RESUMEN DE PAPER · GENERACIÓN DE NIVELES POR RESTRICCIONES

Sturgeon: generación procedural de niveles basados en tiles mediante restricciones aprendidas y diseñadas

Sturgeon: Tile-Based Procedural Level Generation via Learned and Designed Constraints

Autor	Seth Cooper (Northeastern University)

Publicación	Proceedings of the 18th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2022), 18(1), 26–36
Cita APA	Cooper, S. (2022). Sturgeon: Tile-based procedural level generation via learned and designed constraints. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 18*(1), 26–36. https://doi.org/10.1609/aiide.v18i1.21944
Técnica central	Satisfacción de restricciones booleanas (SAT / MaxSAT / ASP / SMT) con restricciones de tiles, patrones, distribución y alcanzabilidad

Resumen

El trabajo presenta **Sturgeon**, un sistema de generación de niveles basados en **tiles** mediante **restricciones**. Propone una pequeña **API de nivel medio** para expresar restricciones, que puede instanciarse con distintos *solvers* de bajo nivel, incluidos *solvers* de portafolio. Con esa API se generan niveles que incorporan restricciones **aprendidas de niveles de ejemplo** y restricciones **dadas por un diseñador**. Además, incluye un enfoque flexible basado en restricciones para **asegurar que la meta del nivel sea alcanzable**. El sistema se demuestra en siete juegos y en aplicaciones como rellenar zonas, reparar niveles y cubrir el rango expresivo.

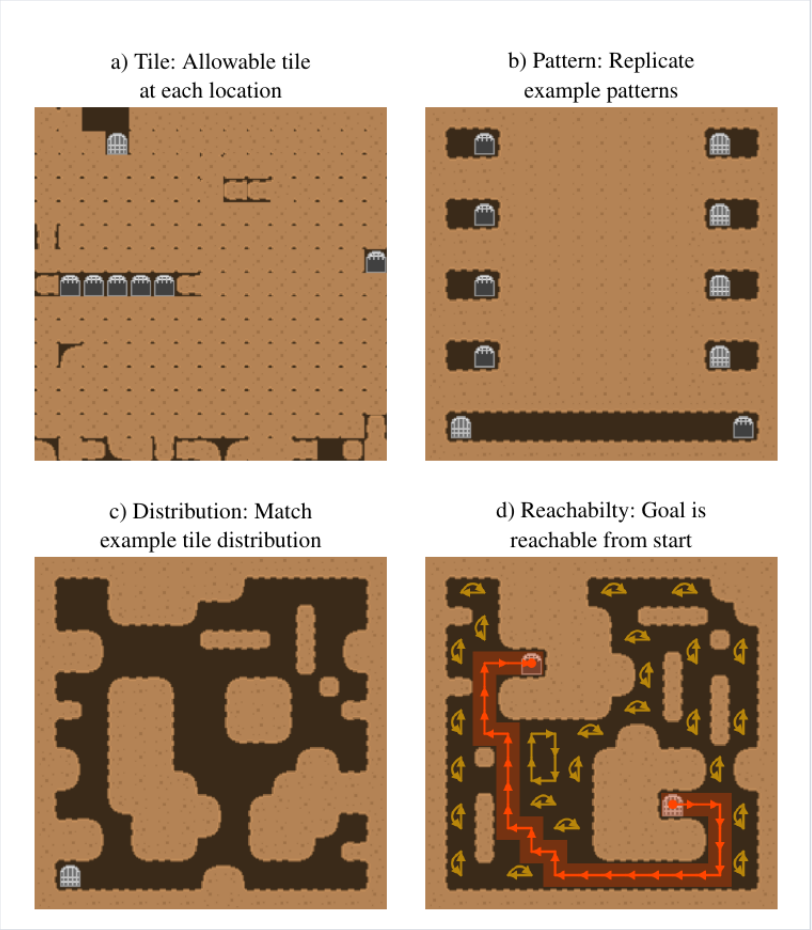


Figura 1. Efecto de agregar cada tipo de regla de diseño, una sobre otra, en el juego de cuevas: (a) tiles: un tile permitido en cada posición; (b) patrones: replicar los patrones del ejemplo; (c) distribución: igualar la distribución de tiles del ejemplo; (d) alcanzabilidad: la meta es alcanzable desde el inicio. El camino de alcanzabilidad se muestra en rojo (los ciclos cerrados no usados, en dorado).

Antecedentes (introducción)

La PCG genera contenido de juegos automáticamente, y la PCG mediante aprendizaje automático (PCGML) aprende a generarlo a partir de ejemplos. Sturgeon es un sistema flexible y eficiente de **PCG basada en restricciones** para niveles 2D de tiles: usa una API pequeña para expresar restricciones sobre **variables booleanas** y las traduce a problemas de satisfacción de restricciones de bajo nivel, que se resuelven con *solvers* estándar (SAT, SMT o Answer Set). Las restricciones pueden **aprenderse** de niveles de ejemplo (patrones y distribuciones de tiles) o **darlas un diseñador** (número o ubicación de ciertos tiles).

El trabajo se apoya en enfoques previos que aprenden patrones de tiles, como **WaveFunctionCollapse**, Model Synthesis o n-gramas, y en trabajos que usan restricciones para garantizar propiedades: WFC implementado con programación de conjuntos de respuestas (Karth y Smith, 2017), laberintos con salidas alcanzables, restricciones en puzles y remuestreo probabilístico. Su aporte es mostrar que *solvers* generales permiten muchos tipos de patrones aprendidos y más control, y formular la **alcanzabilidad de la meta como un problema de grafos** traducible a restricciones, de modo que los niveles se puedan completar con distintas reglas de movimiento del jugador.

Vista general del sistema

El objetivo es generar niveles 2D de tiles mediante restricciones. Un **tile** es una entidad con información **funcional** (de jugabilidad) y/o de **imagen** que puede colocarse en una posición de la grilla; un **nivel** es una grilla 2D. Cada nivel puede tener una grilla funcional (p. ej. tiles sólidos o de meta) y una grilla de imagen (p. ej. ladrillo o bandera). También se usan **etiquetas** (*tags*), que limitan qué tiles pueden ir en cada posición; el nivel existe dentro de una grilla infinita de tiles vacíos (*void*).

Las grillas funcional y de imagen pueden generarse **simultáneamente** o en **secuencia** (primero la funcional y luego la de imagen). La secuencial suele ser más eficiente, porque la alcanzabilidad no depende de cómo se ve un tile (Figura 2).

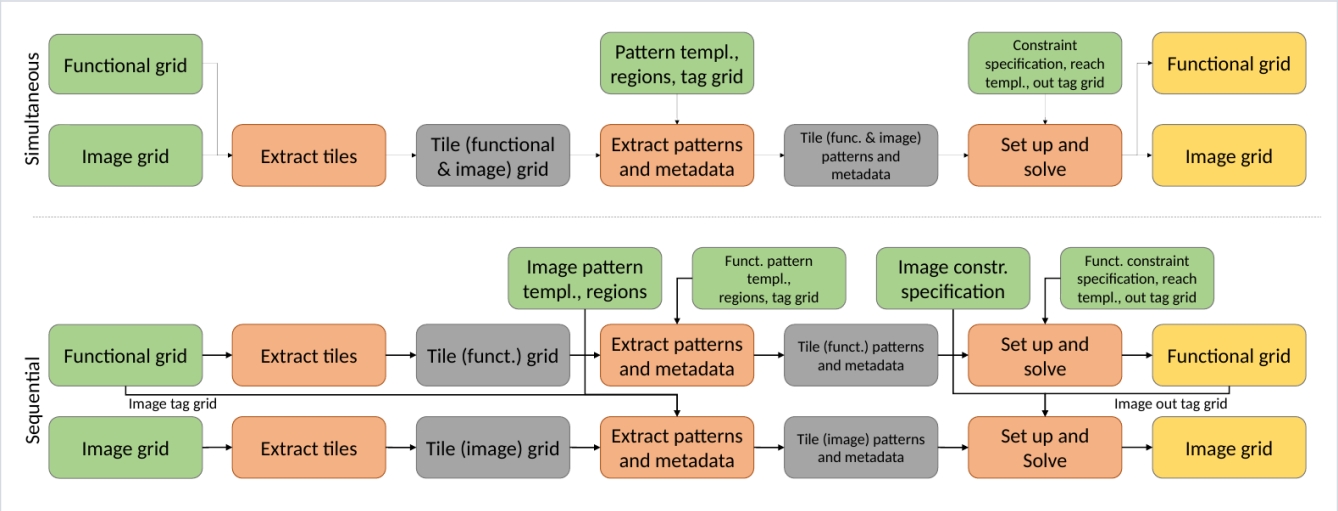


Figura 2. Flujos de aprendizaje y generación, simultáneo (arriba) y secuencial (abajo): se extraen tiles y patrones de los ejemplos y luego se plantea y resuelve el problema.

Se usan **cuatro tipos de reglas de diseño**, todas expresadas con la API:

- **Reglas de tile:** exactamente un tile en cada posición.
- **Reglas de patrón:** relaciones entre tiles cercanos aprendidas de los ejemplos.
- **Reglas de distribución:** preferir que la distribución de tiles de la salida se parezca a la de la entrada.
- **Reglas de alcanzabilidad:** exigir un camino a través del nivel.

El sistema está implementado en Python como un conjunto de scripts de línea de comandos.

API de nivel medio y solvers de bajo nivel

La API ofrece funciones para crear variables booleanas, crear conjunciones, agregar restricciones de **conteo** y de **implicación** (que pueden ser **duras** o **blandas**, con peso), ejecutar el solver y leer los resultados (Tabla 1).

Tabla 1. API de nivel medio y su implementación en los solvers de bajo nivel (traducida y resumida).

Función	Descripción	Implementación (SAT / ASP / SMT)
MAKEVAR()	Crea una variable booleana nueva.	Variable nativa en cada solver.
MAKECONJ(ls)	Crea la conjunción (Y lógico) de los literales dados.	SAT y ASP: nueva variable de conjunción. SMT: *And*.
CNSTRCOUNT(vs, lo, hi, wt)	Restricción: entre *lo* y *hi* de las variables *vs* son verdaderas, con peso *wt*.	SAT: restricciones atMostK (nativas o codificadas). ASP: regla de elección con restricción. SMT: restricciones pseudo-booleanas.
CNSTRIMPLIESDISJ(l, ms, wt)	Restricción: el literal *l* implica la disyunción (O lógico) de *ms*, con peso *wt*.	SAT: cláusula. ASP: regla. SMT: *Implies*, *Or*.
SOLVE()	Ejecuta el solver.	SAT: restricciones blandas nativas. ASP: blandas mediante variables de etiqueta y *#minimize*. SMT: blandas nativas.
GETVAR(v)	Obtiene el valor (verdadero/falso) de una variable.	—
GETOBJECTIVE()	Obtiene la suma de pesos de las restricciones blandas no satisfechas.	—

Soportar varios solvers permite usar **solvers de portafolio**. Los evaluados fueron:

- **clingo-fe y clingo-be** (Answer Set Programming): la versión *frontend* (con *grounding*) resultó lenta; la *backend* construye el programa directamente y evita el *grounding*.
- **z3** (SMT), usando solo variables booleanas.
- **pysat-rc2 y pysat-fm**: solvers **MaxSAT parcial ponderado**, que admiten una mezcla de restricciones duras y blandas con peso, y restricciones de cardinalidad nativas.
- **rc2&be**: portafolio que ejecuta pysat-rc2 y clingo-be en paralelo y usa el primero que encuentra la solución óptima.

Descripción de las reglas de diseño

Reglas de tile

Crean una variable booleana por cada tile permitido en cada posición y exigen que exactamente uno sea verdadero: una codificación tipo ***one-hot***. Una grilla de etiquetas de salida limita los tiles permitidos en cada posición. Con solo estas reglas, los tiles quedan dispuestos al azar (Figura 1a).

Tabla 2. Reglas de tile. *tile* es la variable de un tile individual y *tiles* son las variables de todos los tiles de una posición.

Llamada a la API	Significado
Para cada posición y cada tile posible: MAKEVAR() → tile	Cada posición tiene una variable por cada tile que su etiqueta permite.
Para cada posición: CNSTRCOUNT(tiles, 1, 1, DURA)	Exactamente una variable de tile es verdadera en cada posición.

Reglas de patrón

Hacen que el nivel se parezca al ejemplo replicando patrones locales (Figura 1b). Usan una **plantilla de patrones**, compuesta por plantillas de entrada y salida: la presencia de un patrón de entrada restringe los patrones de salida posibles, y en cada posición debe haber algún patrón de entrada. Los patrones concretos se aprenden de los niveles de ejemplo, y pueden aplicarse en cada posición, por fila, por columna o con un paso (*stride*) arbitrario.

Tabla 3. Reglas de patrón. Se usa una u otra de las dos primeras según haya patrones de salida colocables o no.

Llamada a la API	Significado
MAKECONJ(tilesDelPatrón) → patrón	Un patrón es la conjunción de sus tiles individuales.
CNSTRIMPLIESDISJ(patrónEntrada, patronesSalida, DURA)	Si un patrón de entrada colocable tiene salidas colocables, implica al menos una de ellas.
CNSTRCOUNT(patrónEntrada, 0, 0, DURA)	Si un patrón de entrada colocable no tiene salidas colocables, ese patrón es falso.
CNSTRCOUNT(todosLosPatronesEntrada, 1, ∞, DURA)	En cada posición donde se aplican patrones, al menos un patrón de entrada colocable es verdadero.

Las plantillas usadas (Figura 3) son:

- **nbr-l, nbr-plus:** un tile restringe de forma independiente a algunos de sus vecinos.
- **ring, diamond:** un tile restringe en conjunto a algunos de sus vecinos.
- **3gr-col, 2gr-row:** imitan 3-gramas y 2-gramas (Dahlskog et al., 2014): una columna o fila restringe a la siguiente.
- **zgr-col:** creada para Zelda; n-grama de columnas de 11 tiles (la altura de una sala) que hace coincidir las salas, sobre todo las puertas.
- **no-out3:** entrada de 3×3 sin salida; como en cada posición debe existir un patrón de entrada, **imita WaveFunctionCollapse**.

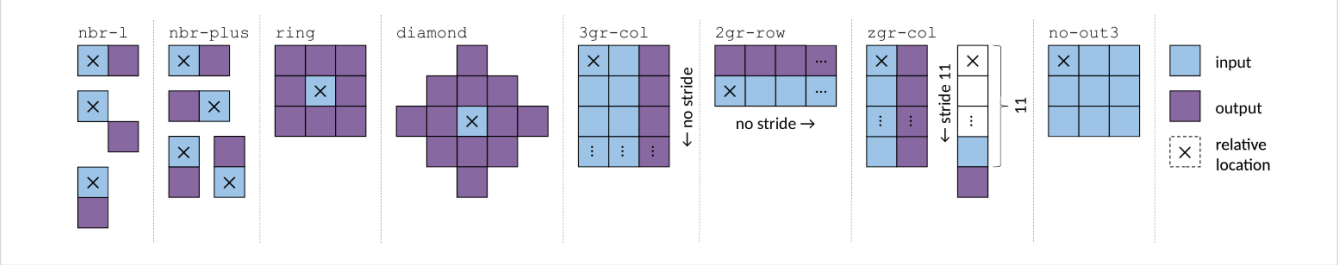


Figura 3. Plantillas de patrones (entrada, salida y posiciones relativas).

Reglas de distribución

Con solo tiles y patrones, la distribución global puede diferir mucho del ejemplo (tiles raros que aparecen a menudo, o al revés). Estas reglas acercan los conteos a los del ejemplo (Figura 1c), por etiqueta y por regiones espaciales: **global** (todo el nivel) o **div** (una grilla gruesa de N×M regiones, que captura p. ej. que las nubes solo aparecen arriba).

Tabla 4. Reglas de distribución. Se usa una restricción **blanda de bajo peso** donde el mínimo y el máximo son ±50 % del número deseado de tiles.

Llamada a la API	Significado
Para cada región, etiqueta y tile: CNSTRCOUNT(tilesContados, mín, máx, BLANDA)	La cantidad de tiles de una región y etiqueta debe parecerse a la deseada según el ejemplo.

Reglas de alcanzabilidad

Las reglas anteriores no consideran el movimiento del jugador, así que el nivel podría ser imposible de completar. Las reglas de alcanzabilidad **aseguran que el nivel sea completable** (Figura 1d). Se basan en un **grafo** construido sobre la grilla: cada posición es un nodo y hay una arista dirigida hacia cada nodo que el jugador podría alcanzar desde ahí. Pero una arista solo puede formar parte del camino si se cumplen condiciones de tiles **abiertos** (transitables) y **cerrados** (no transitables). Por ejemplo, una arista que representa un **salto** solo es usable si la posición bajo el inicio del salto

está cerrada y todas las posiciones del arco del salto están abiertas. Se exige un camino desde el tile de inicio hasta el de meta.

Tabla 5. Reglas de alcanzabilidad (∀ = bucle sobre las variables relevantes; ⊕ = concatenar variables en una lista).

Llamada a la API	Significado
CNSTRCOUNT(iniciosVálidos, 1, 1, DURA); CNSTRCOUNT(metasVálidas, 1, 1, DURA); CNSTRCOUNT(iniciosInválidos, 0, 0, DURA); CNSTRCOUNT(metasInválidas, 0, 0, DURA)	Debe haber exactamente un tile de inicio y uno de meta en posiciones válidas, y ninguno en posiciones inválidas.
Para cada arista: MAKEVAR() → arista	Cada arista tiene una variable que indica si es alcanzable.
Para cada nodo: MAKEVAR() → nodo	Cada nodo tiene una variable que indica si es alcanzable.
MAKEVAR() → abierto; ∀tileAbierto CNSTRIMPLIESDISJ(tileAbierto, abierto, DURA); CNSTRIMPLIESDISJ(abierto, tilesAbiertos, DURA)	Cada nodo tiene una variable «abierto», verdadera si y solo si su tile es un tile abierto.
∀aristaSaliente CNSTRIMPLIESDISJ(¬nodo, ¬aristaSaliente, DURA)	Si un nodo no es alcanzable, ninguna de sus aristas salientes lo es.
∀aristaSaliente ∀necesitaAbierto CNSTRIMPLIESDISJ(¬abierto, ¬arista, DURA); ∀necesitaCerrado CNSTRIMPLIESDISJ(abierto, ¬arista, DURA)	Si una posición que debe estar abierta está cerrada (o al revés), la arista no es alcanzable.
CNSTRCOUNT(aristasEntrantes, 0, 1, DURA); CNSTRCOUNT(aristasSalientes, 0, 1, DURA)	Un nodo tiene a lo más una arista entrante alcanzable y una saliente alcanzable.
MAKECONJ(¬tileInicio ⊕ ∀¬aristaEntrante) → sinEntrada; CNSTRIMPLIESDISJ(sinEntrada, ¬nodo, DURA)	Un nodo que no es el inicio y no tiene aristas entrantes alcanzables no es alcanzable.
CNSTRIMPLIESDISJ(tileInicio, nodo, DURA); ∀aristaEntrante CNSTRIMPLIESDISJ(tileInicio, ¬aristaEntrante, DURA)	El nodo del inicio es alcanzable y no tiene aristas entrantes alcanzables.
CNSTRIMPLIESDISJ(tileMeta, nodo, DURA); ∀aristaSaliente CNSTRIMPLIESDISJ(tileMeta, ¬aristaSaliente, DURA)	El nodo de la meta es alcanzable y no tiene aristas salientes alcanzables.

El enfoque se parece al de Aloul et al. (encontrar caminos con SAT), pero ese trabajo usa grafos no dirigidos, con todas las aristas siempre usables, y optimiza para el camino más corto. Dos efectos secundarios interesantes: (1) solo se exige que **exista** un camino, no que sea corto o directo, así que puede ser muy rebuscado; y (2) además del camino, pueden aparecer **ciclos cerrados** fuera de él.

Las **plantillas de alcanzabilidad** definen las aristas y sus condiciones (Figura 4):

- **maze:** el jugador se mueve en 4 direcciones a tiles abiertos adyacentes (cuevas, Zelda).
- **tomb:** se mueve en 4 direcciones, pero sigue avanzando hasta que un tile cerrado lo detiene.
- **platform:** plantilla básica de plataformas, basada en Summerville et al. (2015): el jugador puede **caer** por tiles abiertos y **caminar o saltar** desde encima de tiles cerrados. Se usa en Super Mario Bros., Super Mario Land y Kid Icarus (que además permite pasar de un borde lateral al otro).
- **supercat:** el jugador puede caer y caminar, además de trepar paredes, saltar desde paredes y saltar desde bordes; no hay salto «estándar». Es una simplificación del movimiento real.

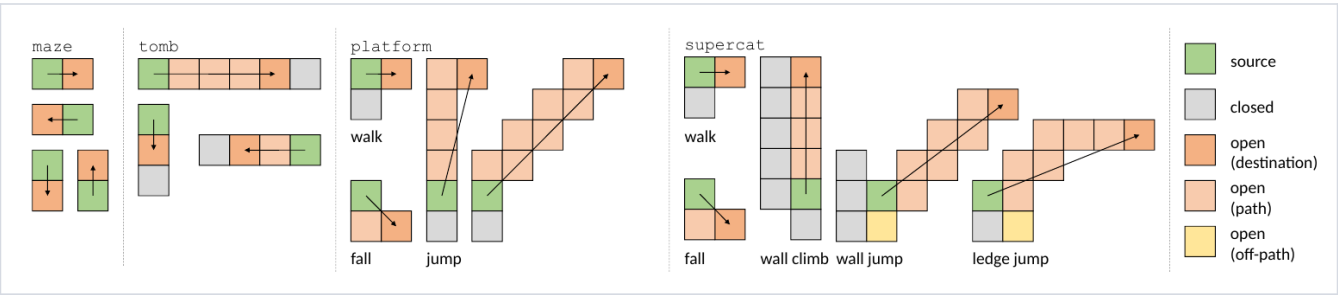


Figura 4. Ejemplos de plantillas de alcanzabilidad (no es una lista completa): maze, tomb, platform (caminar, caer, saltar) y supercat (caminar, caer, trepar, saltar desde pared o borde). Colores: origen, cerrado, abierto (destino), abierto (camino) y abierto (fuera del camino).

Juegos

Tabla A. Juegos usados en el trabajo (elaboración del resumen a partir del texto).

Nombre	Juego	Origen de los patrones
cave	Mapa de cuevas visto desde arriba, creado para el trabajo	Ejemplo propio (tiles de Kenney)
tomb	Tomb of the Mask: laberinto donde el jugador avanza hasta chocar	Nivel 1
zelda	The Legend of Zelda (1986)	Mazmorra 1-1 del VGLC
icarus	Kid Icarus (1987)	Nivel 1 del VGLC
mariobros	Super Mario Bros. (1985)	Nivel 1-1 del VGLC
marioland	Super Mario Land (1989)	Nivel 1-1 del VGLC
supercat	Super Cat Tales: plataformas sin salto estándar	Sección de árboles del nivel 1-7

Nota del resumen: el paper indica 1983 como año de Super Mario Bros.; el juego se lanzó en 1985.

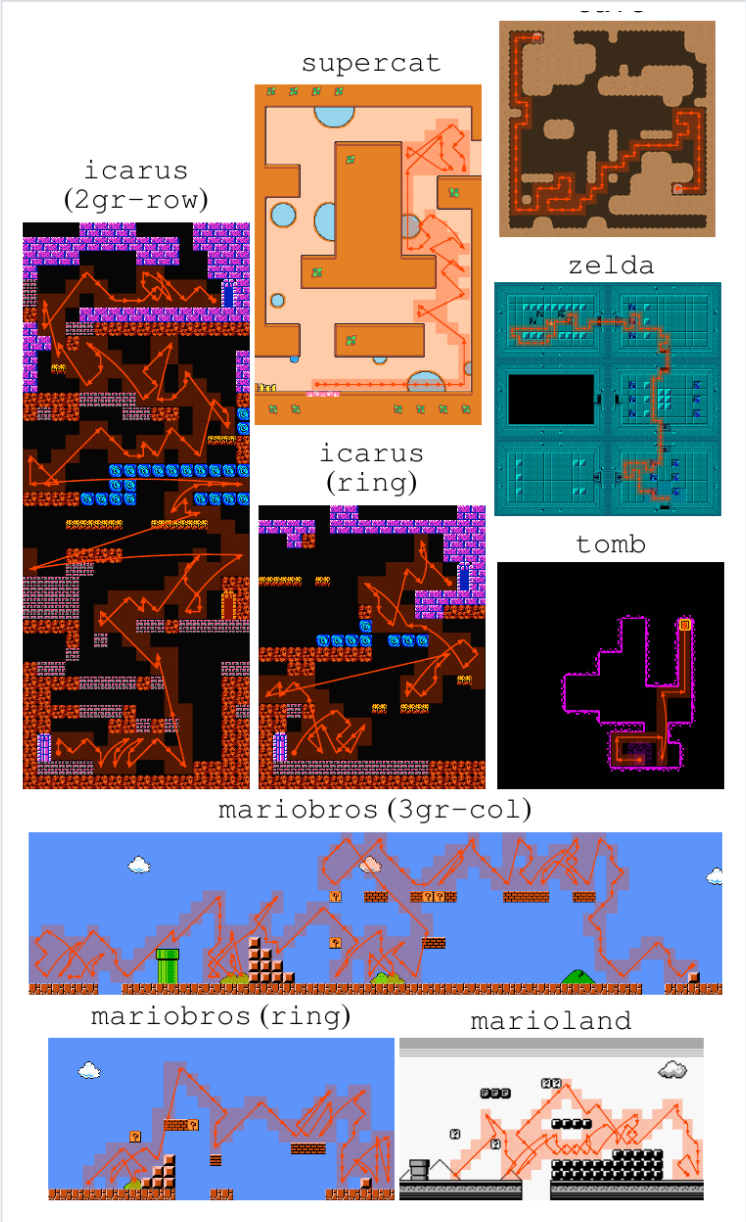


Figura 5. Ejemplos de niveles generados para cada juego. El camino de alcanzabilidad se muestra en rojo. Más ejemplos en el Apéndice A.

Comparación de solvers y generación de niveles

Para cada configuración (juego, flujo, plantillas, regiones, tamaño y solver) se generaron **25 niveles** y se midieron el **tiempo de generación** y el **rango** (fracción de tiles distintos entre todos los pares de niveles). Se buscaron tamaños que la mayoría de los solvers resolviera en unos 10 segundos o menos.

- **Niveles pequeños** (cave, icarus, mariobros): clingo-fe y z3 fueron mucho más lentos, y pysat-fm en general más lento que pysat-rc2, por lo que se descartaron. El rango casi no varió entre solvers.
- **Niveles grandes:** clingo-be fue más rápido en cave y pysat-rc2 en icarus, pero el **portafolio rc2&be** rindió bien en todos, así que se adoptó desde ahí.
- **Todos los juegos:** se observan algunos defectos. Con n-gramas en icarus hay poca variación (el inicio y el final quedan casi memorizados); en cave y tomb aparecen zonas inalcanzables; en marioland y supercat hay pirámides incompletas y ventanas no circulares.

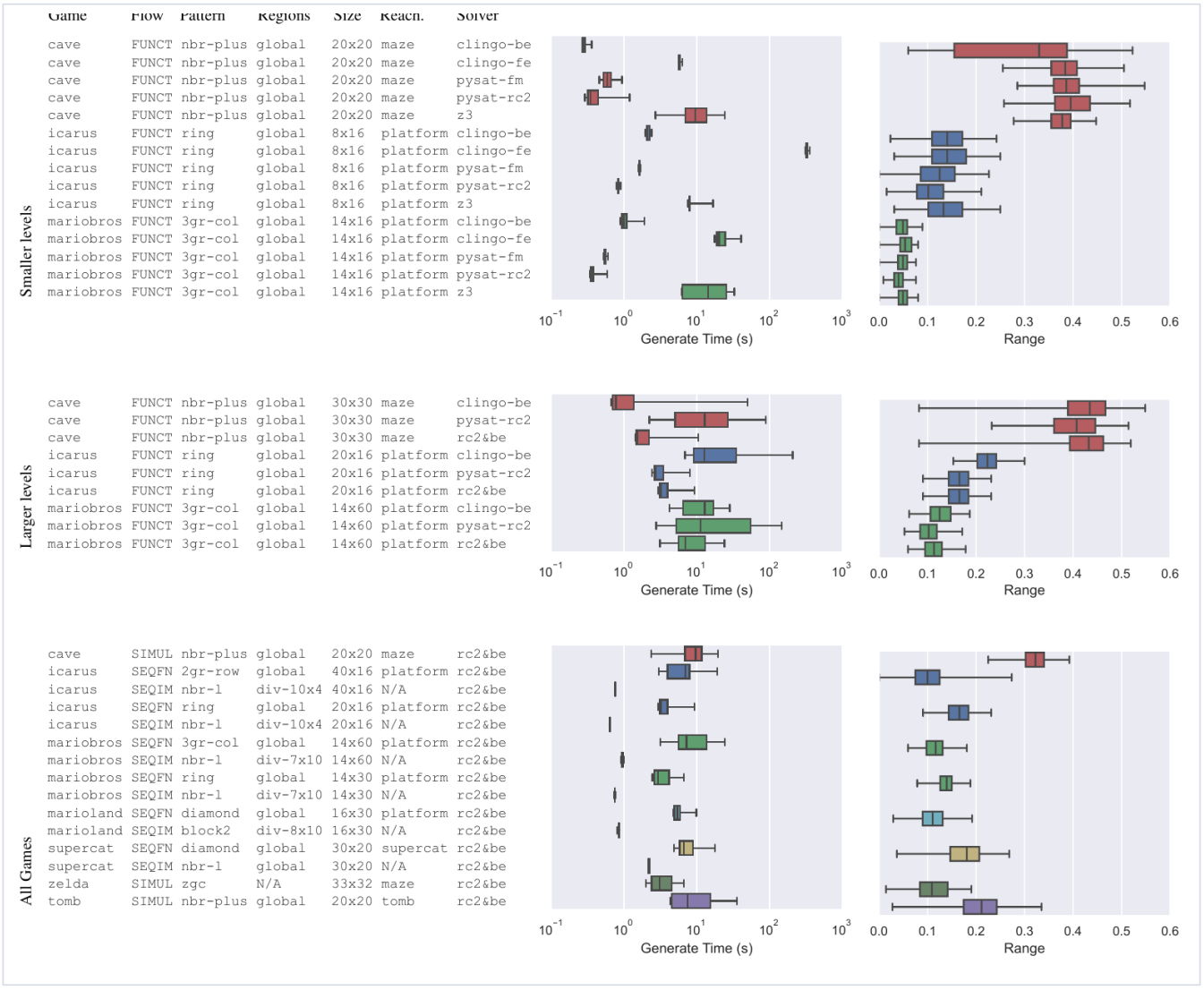


Figura 6. Tiempos de generación (preparación y resolución) y rangos: niveles pequeños (arriba), niveles más grandes (medio) y todos los juegos (abajo).

Aplicaciones de controlabilidad

Se muestran aplicaciones que aprovechan las restricciones del diseñador (5 niveles o imágenes por ejemplo). Las primeras usan la configuración mariobros (ring):

Tabla B. Aplicaciones de controlabilidad y tiempos funcionales (mediana / máximo).

Aplicación	Qué se pide	Tiempo
tall-pipe	Nivel 14×24 con exactamente una tubería que llegue a la 3.ª fila desde arriba.	3,1 s / 3,8 s
3?	Nivel 14×24 con exactamente 3 bloques «?» (no necesariamente alcanzables).	2,0 s / 2,1 s
max-gap	Nivel 14×24 que **maximiza** los tiles vacíos en la fila inferior (huecos).	2,5 s / 3,7 s
void	Grilla 16×40 con etiquetas que solo permiten tiles vacíos, para recortar la forma del nivel.	3,1 s / 25,2 s
infill	Grilla 14×32 fija salvo una zona central vacía que hay que rellenar (útil en herramientas cocreativas).	2,6 s / 3,0 s
link	Grilla 14×37 con dos segmentos fijos y un espacio entre ellos para **unirlos**.	3,0 s / 3,1 s
repair	Nivel 14×18 incompletable (tubería incompleta): el solver hace **cambios mínimos** para repararlo y hacerlo completable.	10,5 min / 24,8 min
flowers-1/7, skyline-15/20	Ejemplos de WFC (solo imagen) con el patrón no-out3, más restricciones de número de flores (1 y 7) y ventanas (15 y 20).	30,3 s; 3,6 s; 6,9 s; 6,0 s (mediana)

En *repair*, aunque se encontraron caminos distintos, todas las soluciones agregaron el bloque en el mismo lugar. Los ejemplos tipo WFC muestran el **control adicional** que un solver de restricciones da sobre el enfoque estándar de patrones de WFC.

Cobertura del rango expresivo

El análisis habitual del rango expresivo genera muchos niveles y luego los mide (linealidad, indulgencia; Smith y Whitehead, 2010), y la búsqueda de calidad-diversidad (MAP-Elites) optimiza poblaciones según características similares. Con Sturgeon, en cambio, se puede **restringir el rango permitido** de una propiedad y luego generar un nivel. Con mariobros (ring) se exploraron dos dimensiones, **número de tiles de hueco** y **número de tiles sólidos**, con 6 rangos cada una y un límite de 10 minutos por nivel de 14×24: se encontraron **19 de 36** niveles posibles; la mayoría de los faltantes requerían muy pocos o muchos tiles sólidos, y 7 agotaron el tiempo. La generación total tomó 86,2 minutos (2,4 min por nivel posible).

Conclusión

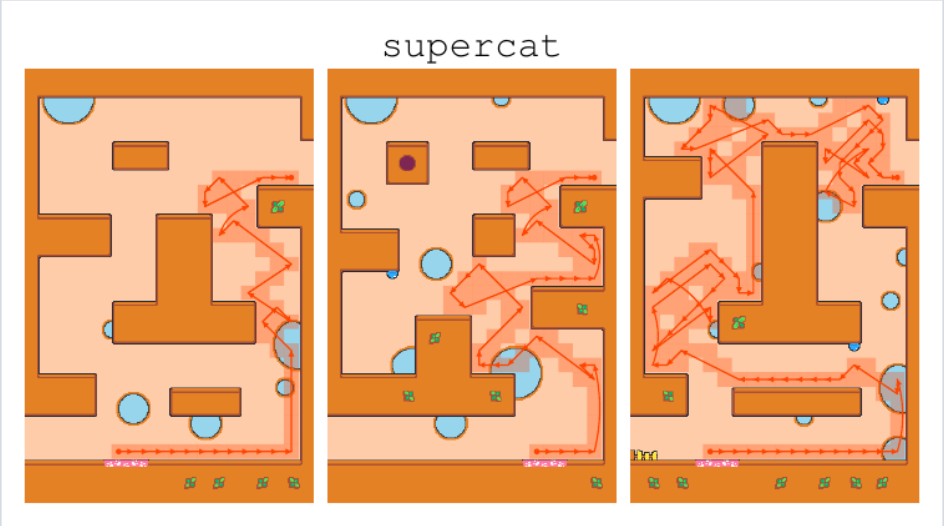
El trabajo presenta un sistema con una API de restricciones de nivel medio para generar niveles 2D de tiles, demostrado en varios juegos y aplicaciones, y evalúa su rendimiento y la controlabilidad que ofrece. Como trabajo futuro, el autor propone extenderlo a **grillas 3D** y otras estructuras (grafos), soportar **metas secuenciales** y mecánicas como llave-y-cerradura, y mejorar el rendimiento, sobre todo en la reparación de niveles.

Utilidad para tu modelo matemático

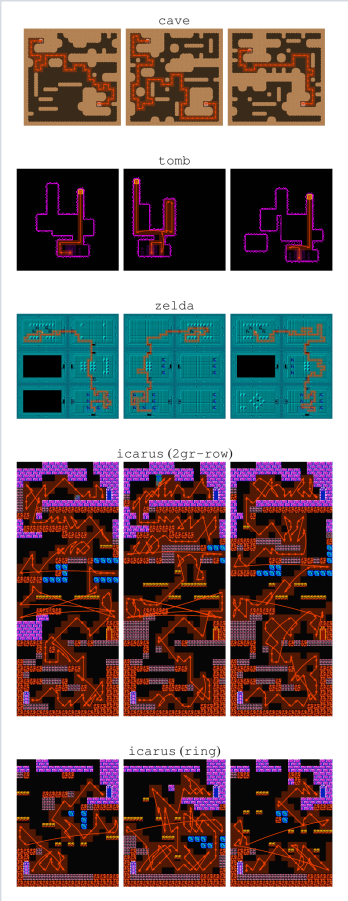
Es la referencia más directa para formular el problema como **CSP/CSOP** en plataformas 2D:

- **Variables:** una booleana por tile posible en cada posición, con exactamente una verdadera (*one-hot*), equivalente a $X_{ij} \in T$.
- **Restricciones duras:** unicidad por celda, patrones locales, un inicio y una meta, y **alcanzabilidad** mediante variables de nodo y arista con plantillas de movimiento (caminar, caer, saltar).
- **Restricciones blandas con peso:** distribución de tiles (±50 %); el solver minimiza la suma de pesos incumplidos (GETOBJECTIVE). Con ellas, el problema pasa de CSP a **optimización con restricciones** (MaxSAT ponderado).
- **Limitaciones:** el camino no tiene por qué ser corto ni directo; los niveles son pequeños (p. ej. 14×60); la reparación es lenta; algunos defectos visuales y zonas inalcanzables.

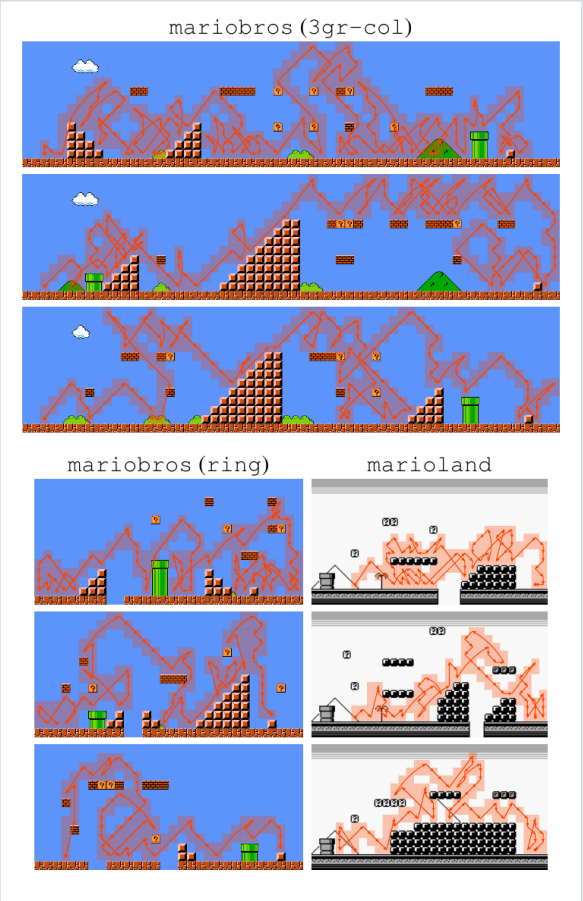
Apéndice A: imágenes de niveles



Apéndice A (1). Niveles generados para supercat.

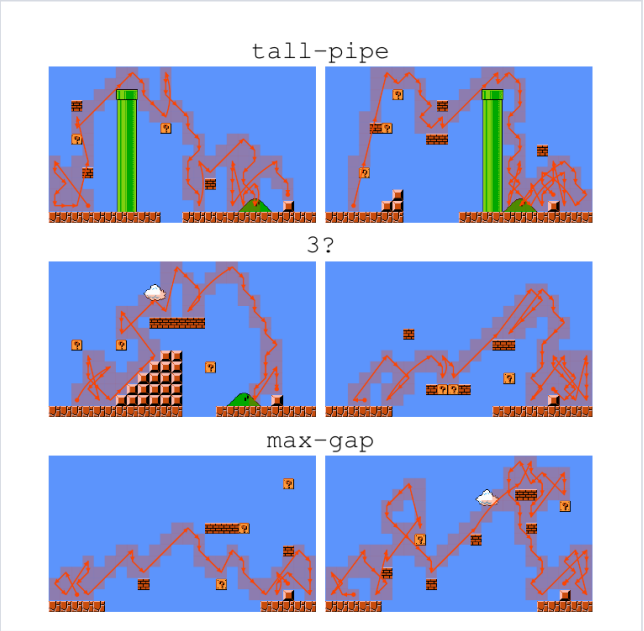


Apéndice A (2). cave, tomb, zelda, icarus (2gr-row) e icarus (ring).

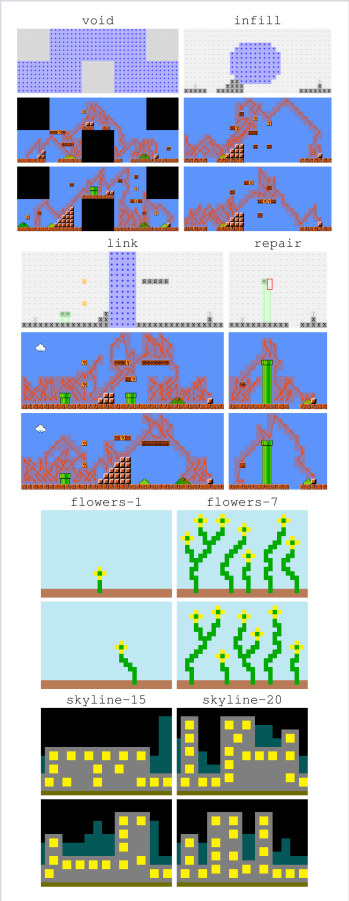


Apéndice A (3). mariobros (3gr-col), mariobros (ring) y marioland.

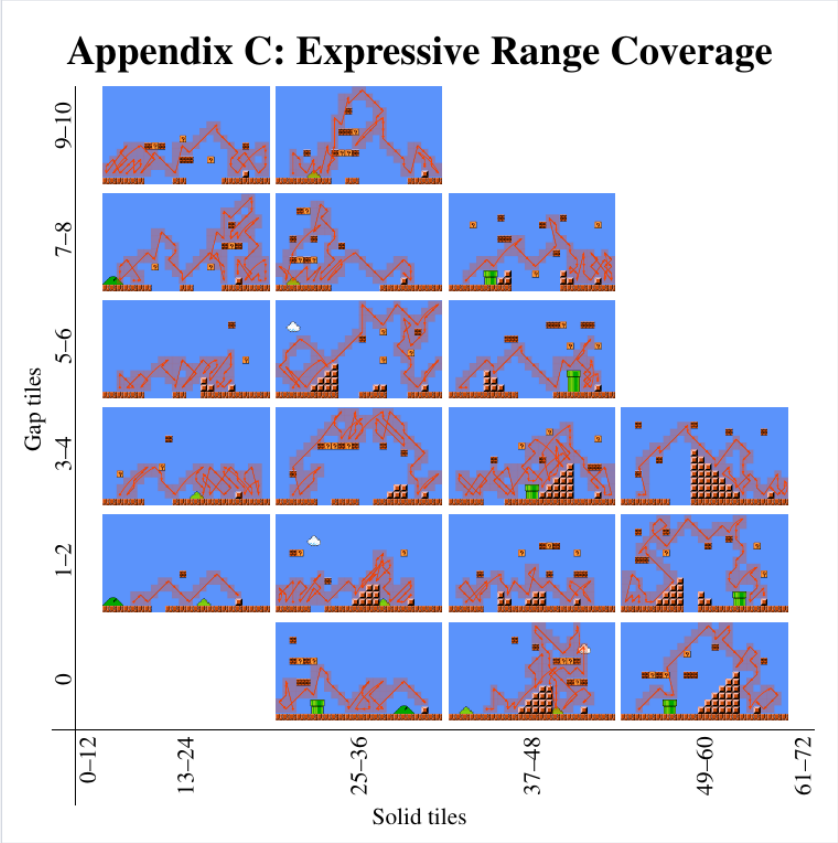
Apéndices B y C: aplicaciones y cobertura del rango expresivo



Apéndice B (1). Aplicaciones tall-pipe, 3? y max-gap.



Apéndice B (2). void, infill, link, repair y ejemplos de WFC (flowers, skyline).



Apéndice C. Cobertura del rango expresivo: niveles encontrados según tiles de hueco (eje vertical) y tiles sólidos (eje horizontal).